

Unit 4.1

Assembly Language Programming – Part 2

CpE 3104 - Microprocessors

8086 supports 6 types of instructions:

1. **Data Transfer Instructions**
2. **Arithmetic Instructions**
3. **Logical Instructions**
4. **String manipulation Instructions**
5. **Process Control Instructions**
6. **Control Transfer Instructions**

Data Transfer Instructions

1. DATA TRANSFER Instructions

Instructions that are used to transfer data/ address in to registers, memory locations and I/O ports.

Generally involve two operands: **Source operand** and **Destination operand** of the same size.

Source: Register or a memory location or an immediate data

Destination : Register or a memory location.

The size should be either a byte or a word.

A 8-bit data can only be moved to 8-bit register/ memory and a 16-bit data can be moved to 16-bit register/ memory.

1. DATA TRANSFER Instructions

Mnemonics: **MOV, XCHG, PUSH, POP, IN, OUT ...**

MOV reg2/ mem, reg1/ mem

MOV reg2, reg1
MOV mem, reg1
MOV reg2, mem

$(reg2) \leftarrow (reg1)$
 $(mem) \leftarrow (reg1)$
 $(reg2) \leftarrow (mem)$

MOV reg/ mem, data

MOV reg, data
MOV mem, data

$(reg) \leftarrow data$
 $(mem) \leftarrow data$

XCHG reg2/ mem, reg1

XCHG reg2, reg1
XCHG mem, reg1

$(reg2) \leftrightarrow (reg1)$
 $(mem) \leftrightarrow (reg1)$

1. DATA TRANSFER Instructions

Mnemonics: **MOV, XCHG, PUSH, POP, IN, OUT ...**

PUSH reg16/ mem

PUSH reg16

$$\begin{aligned} (SP) &\leftarrow (SP) - 2 \\ MA_S &= (SS) \times 16_{10} + SP \\ (MA_S; MA_S + 1) &\leftarrow (reg16) \end{aligned}$$

PUSH mem

$$\begin{aligned} (SP) &\leftarrow (SP) - 2 \\ MA_S &= (SS) \times 16_{10} + SP \\ (MA_S; MA_S + 1) &\leftarrow (mem) \end{aligned}$$

POP reg16/ mem

POP reg16

$$\begin{aligned} MA_S &= (SS) \times 16_{10} + SP \\ (reg16) &\leftarrow (MA_S; MA_S + 1) \\ (SP) &\leftarrow (SP) + 2 \end{aligned}$$

POP mem

$$\begin{aligned} MA_S &= (SS) \times 16_{10} + SP \\ (mem) &\leftarrow (MA_S; MA_S + 1) \\ (SP) &\leftarrow (SP) + 2 \end{aligned}$$

1. DATA TRANSFER Instructions

Mnemonics: **MOV, XCHG, PUSH, POP, IN, OUT, ...**

IN A, [DX] IN AL, [DX] IN AX, [DX]	$PORT_{addr} = (DX)$ $(AL) \leftarrow (PORT)$ $PORT_{addr} = (DX)$ $(AX) \leftarrow (PORT)$	OUT [DX], A OUT [DX], AL OUT [DX], AX	$PORT_{addr} = (DX)$ $(PORT) \leftarrow (AL)$ $PORT_{addr} = (DX)$ $(PORT) \leftarrow (AX)$
IN A, addr8 IN AL, addr8 IN AX, addr8	$(AL) \leftarrow (addr8)$ $(AX) \leftarrow (addr8)$	OUT addr8, A OUT addr8, AL OUT addr8, AX	$(addr8) \leftarrow (AL)$ $(addr8) \leftarrow (AX)$

Arithmetic Instructions

2. ARITHMETIC Instructions

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, DIV, CMP...**

ADD reg2/ mem, reg1/mem

ADC reg2, reg1
ADC reg2, mem
ADC mem, reg1

$(reg2) \leftarrow (reg1) + (reg2)$
 $(reg2) \leftarrow (reg2) + (mem)$
 $(mem) \leftarrow (mem) + (reg1)$

ADD reg/mem, data

ADD reg, data
ADD mem, data

$(reg) \leftarrow (reg) + data$
 $(mem) \leftarrow (mem) + data$

ADD A, data

ADD AL, data8
ADD AX, data16

$(AL) \leftarrow (AL) + data8$
 $(AX) \leftarrow (AX) + data16$

2. ARITHMETIC Instructions

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, DIV, CMP...**

ADC reg2/ mem, reg1/mem ADC reg2, reg1 ADC reg2, mem ADC mem, reg1	$(\text{reg2}) \leftarrow (\text{reg1}) + (\text{reg2}) + \text{CF}$ $(\text{reg2}) \leftarrow (\text{reg2}) + (\text{mem}) + \text{CF}$ $(\text{mem}) \leftarrow (\text{mem}) + (\text{reg1}) + \text{CF}$
ADC reg/mem, data ADC reg, data ADC mem, data	$(\text{reg}) \leftarrow (\text{reg}) + \text{data} + \text{CF}$ $(\text{mem}) \leftarrow (\text{mem}) + \text{data} + \text{CF}$
ADDC A, data ADD AL, data8 ADD AX, data16	$(\text{AL}) \leftarrow (\text{AL}) + \text{data8} + \text{CF}$ $(\text{AX}) \leftarrow (\text{AX}) + \text{data16} + \text{CF}$

2. ARITHMETIC Instructions

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, DIV, CMP...**

SUB reg2/ mem, reg1/mem

SUB reg2, reg1
SUB reg2, mem
SUB mem, reg1

$(reg2) \leftarrow (reg1) - (reg2)$
 $(reg2) \leftarrow (reg2) - (mem)$
 $(mem) \leftarrow (mem) - (reg1)$

SUB reg/mem, data

SUB reg, data
SUB mem, data

$(reg) \leftarrow (reg) - data$
 $(mem) \leftarrow (mem) - data$

SUB A, data

SUB AL, data8
SUB AX, data16

$(AL) \leftarrow (AL) - data8$
 $(AX) \leftarrow (AX) - data16$

2. ARITHMETIC Instructions

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, DIV, CMP...**

SBB reg2/ mem, reg1/mem

SBB reg2, reg1
SBB reg2, mem
SBB mem, reg1

$(reg2) \leftarrow (reg1) - (reg2) - CF$
 $(reg2) \leftarrow (reg2) - (mem) - CF$
 $(mem) \leftarrow (mem) - (reg1) - CF$

SBB reg/mem, data

SBB reg, data
SBB mem, data

$(reg) \leftarrow (reg) - data - CF$
 $(mem) \leftarrow (mem) - data - CF$

SBB A, data

SBB AL, data8
SBB AX, data16

$(AL) \leftarrow (AL) - data8 - CF$
 $(AX) \leftarrow (AX) - data16 - CF$

2. ARITHMETIC Instructions

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, DIV, CMP...**

INC reg/ mem

INC reg8

$(\text{reg8}) \leftarrow (\text{reg8}) + 1$

INC reg16

$(\text{reg16}) \leftarrow (\text{reg16}) + 1$

INC mem

$(\text{mem}) \leftarrow (\text{mem}) + 1$

DEC reg/ mem

DEC reg8

$(\text{reg8}) \leftarrow (\text{reg8}) - 1$

DEC reg16

$(\text{reg16}) \leftarrow (\text{reg16}) - 1$

DEC mem

$(\text{mem}) \leftarrow (\text{mem}) - 1$

2. ARITHMETIC Instructions

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, IMUL, DIV, CMP...**

MUL reg/ mem	
MUL reg	<u>For byte</u> : $(AX) \leftarrow (AL) \times (\text{reg8})$ <u>For word</u> : $(DX)(AX) \leftarrow (AX) \times (\text{reg16})$
MUL mem	<u>For byte</u> : $(AX) \leftarrow (AL) \times (\text{mem8})$ <u>For word</u> : $(DX)(AX) \leftarrow (AX) \times (\text{mem16})$
IMUL reg/ mem	
IMUL reg	<u>For byte</u> : $(AX) \leftarrow (AL) \times (\text{reg8})$ <u>For word</u> : $(DX)(AX) \leftarrow (AX) \times (\text{reg16})$
IMUL mem	<u>For byte</u> : $(AX) \leftarrow (AX) \times (\text{mem8})$ <u>For word</u> : $(DX)(AX) \leftarrow (AX) \times (\text{mem16})$

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, IMUL, DIV, IDIV, CMP...**

DIV reg/ mem

DIV reg

For 16-bit :- 8-bit :

$(AL) \leftarrow (AX) \div (\text{reg8})$ Quotient

$(AH) \leftarrow (AX) \bmod (\text{reg8})$ Remainder

For 32-bit :- 16-bit :

$(AX) \leftarrow (DX)(AX) \div (\text{reg16})$ Quotient

$(DX) \leftarrow (DX)(AX) \bmod (\text{reg16})$ Remainder

DIV mem

For 16-bit :- 8-bit :

$(AL) \leftarrow (AX) \div (\text{mem8})$ Quotient

$(AH) \leftarrow (AX) \bmod (\text{mem8})$ Remainder

For 32-bit :- 16-bit :

$(AX) \leftarrow (DX)(AX) \div (\text{mem16})$ Quotient

$(DX) \leftarrow (DX)(AX) \bmod (\text{mem16})$ Remainder

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, IMUL, DIV, IDIV, CMP...**

IDIV reg/ mem

IDIV reg

For 16-bit :- 8-bit :

$(AL) \leftarrow (AX) :- (reg8)$ Quotient

$(AH) \leftarrow (AX) \text{ MOD}(reg8)$ Remainder

For 32-bit :- 16-bit :

$(AX) \leftarrow (DX)(AX) :- (reg16)$ Quotient

$(DX) \leftarrow (DX)(AX) \text{ MOD}(reg16)$ Remainder

IDIV mem

For 16-bit :- 8-bit :

$(AL) \leftarrow (AX) :- (mem8)$ Quotient

$(AH) \leftarrow (AX) \text{ MOD}(mem8)$ Remainder

For 32-bit :- 16-bit :

$(AX) \leftarrow (DX)(AX) :- (mem16)$ Quotient

$(DX) \leftarrow (DX)(AX) \text{ MOD}(mem16)$ Remainder

Mnemonics: **ADD, ADC, SUB, SBB, INC, DEC, MUL, IMUL, DIV, IDIV, CMP...**

CMP reg2/mem, reg1/ mem

CMP reg2, reg1

Modify flags $\leftarrow$ (reg2) - (reg1)

If (reg2) > (reg1) then CF=0, ZF=0, SF=0

If (reg2) < (reg1) then CF=1, ZF=0, SF=1

If (reg2) = (reg1) then CF=0, ZF=1, SF=0

CMP reg2, mem

Modify flags $\leftarrow$ (reg2) - (mem)

If (reg2) > (mem) then CF=0, ZF=0, SF=0

If (reg2) < (mem) then CF=1, ZF=0, SF=1

If (reg2) = (mem) then CF=0, ZF=1, SF=0

CMP mem, reg1

Modify flags $\leftarrow$ (mem) - (reg1)

If (mem) > (reg1) then CF=0, ZF=0, SF=0

If (mem) < (reg1) then CF=1, ZF=0, SF=1

If (mem) = (reg1) then CF=0, ZF=1, SF=0

Logical Instructions

3. LOGICAL Instructions

Mnemonics: **AND, OR, XOR, TEST, CMP**

AND A, data

AND AL, data8

AND AX, data16

$(AL) \leftarrow (AL) \& \text{data8}$

$(AX) \leftarrow (AL) \& \text{data16}$

AND reg/ mem, data

AND reg, data

AND mem, data

$(\text{reg}) \leftarrow (\text{reg}) \& \text{data}$

$(\text{mem}) \leftarrow (\text{mem}) \& \text{data}$

3. LOGICAL Instructions

Mnemonics: **AND, OR, XOR, TEST, CMP**

OR reg2/ mem, reg1/mem OR reg2, reg1 OR reg2, mem OR mem, reg1	$(reg2) \leftarrow (reg1) \mid (reg2)$ $(reg2) \leftarrow (reg2) \mid (mem)$ $(mem) \leftarrow (mem) \mid (reg1)$
OR reg/mem, data OR reg, data OR mem, data	$(reg) \leftarrow (reg) \mid data$ $(mem) \leftarrow (mem) \mid data$
OR A, data OR AL, data8 OR AX, data16	$(AL) \leftarrow (AL) \mid data8$ $(AX) \leftarrow (AX) \mid data16$

3. LOGICAL Instructions

Mnemonics: **AND, OR, XOR, TEST, CMP**

XOR reg2/ mem, reg1/mem XOR reg2, reg1 XOR reg2, mem XOR mem, reg1	$(reg2) \leftarrow (reg1) \wedge (reg2)$ $(reg2) \leftarrow (reg2) \wedge (mem)$ $(mem) \leftarrow (mem) \wedge (reg1)$
XOR reg/mem, data XOR reg, data XOR mem, data	$(reg) \leftarrow (reg) \wedge data$ $(mem) \leftarrow (mem) \wedge data$
XOR A, data XOR AL, data8 XOR AX, data16	$(AL) \leftarrow (AL) \wedge data8$ $(AX) \leftarrow (AX) \wedge data16$

3. LOGICAL Instructions

Mnemonics: **AND, OR, XOR, TEST, CMP**

TEST reg2/ mem, reg1/mem TEST reg2, reg1 TEST reg2, mem TEST mem, reg1	Modify flags $\leftarrow$ (reg1) & (reg2) Modify flags $\leftarrow$ (reg2) & (mem) Modify flags $\leftarrow$ (mem) & (reg1)
TEST reg/mem, data TEST reg, data TEST mem, data	Modify flags $\leftarrow$ (reg) & data Modify flags $\leftarrow$ (mem) & data
TEST A, data TEST AL, data8 TEST AX, data16	Modify flags $\leftarrow$ (AL) & data8 Modify flags $\leftarrow$ (AX) * data16

3. LOGICAL Instructions

Mnemonics: **AND, OR, XOR, TEST, CMP**

TEST reg2/ mem, reg1/mem TEST reg2, reg1 TEST reg2, mem TEST mem, reg1	Modify flags $\leftarrow$ (reg1) & (reg2) Modify flags $\leftarrow$ (reg2) & (mem) Modify flags $\leftarrow$ (mem) & (reg1)
TEST reg/mem, data TEST reg, data TEST mem, data	Modify flags $\leftarrow$ (reg) & data Modify flags $\leftarrow$ (mem) & data
TEST A, data TEST AL, data8 TEST AX, data16	Modify flags $\leftarrow$ (AL) & data8 Modify flags $\leftarrow$ (AX) * data16

3. LOGICAL Instructions

Mnemonics: **AND, OR, XOR, TEST, CMP**

CMP reg/mem, data

CMP reg, data

Modify flags $\leftarrow$ (reg) - (data)

If (reg) > data then CF=0, ZF=0, SF=0

If (reg) < data then CF=1, ZF=0, SF=1

If (reg) = data then CF=0, ZF=1, SF=0

CMP mem, data

Modify flags $\leftarrow$ (mem) - (mem)

If (mem) > data then CF=0, ZF=0, SF=0

If (mem) < data then CF=1, ZF=0, SF=1

If (mem) = data then CF=0, ZF=1, SF=0

3. LOGICAL Instructions

Mnemonics: **AND, OR, XOR, TEST, CMP**

CMP A, data

CMP AL, data8

CMP AX, data16

Modify flags $\leftarrow$ (AL) - data8

If (AL) > data8 then CF=0, ZF=0, SF=0

If (AL) < data8 then CF=1, ZF=0, SF=1

If (AL) = data8 then CF=0, ZF=1, SF=0

Modify flags $\leftarrow$ (AX) - data16

If (AX) > data16 then CF=0, ZF=0, SF=0

If (mem) < data16 then CF=1, ZF=0, SF=1

If (mem) = data16 then CF=0, ZF=1, SF=0